Summary Introduction to Logic

University of Groningen

Tom Oostveen

2026

Contents

1 Validity of arguments 1
2 Fitch proofs 1
2.1 Boolean connectives 2
2.2 Negation 3
2.3 (bi-)implication 3
2.4 Quantifiers 5
3 Logical Equivalences and laws 5
4 Normal forms 6
4.1 Negation normal form (NNF) 6
4.2 Conjunction normal form (CNF) 6
4.3 Disjunction normal form (DNF) 6
4.4 Quantifier normal forms 6
5 Models 8
5.1 Interpretations 8
5.2 Variable assignments 9
5.3 Proving satisfiability in a model 9

1 Validity of arguments

An argument consists of zero or more premises, and a conclusion. An argument is *valid* if the conclusion always follows from the premises. (assuming the premises are true)

An argument is *sound* if the premises are also confirmed to be true.

An argument is *valid* if there's no possible *counterexample* to the argument. A *counterexample* (in)formally describes a situation in which the premises are true, but the conclusion is false. In the midterm, you'll use an informal counterexample. In the final exam, you'll be writing a *countermodel*. See the section about models.

2 Fitch proofs

I have a big ass repo of example fitch proofs, you can use it either as an LPL answer key or as a way to learn how to go about certain proofs: https://github.com/dtomvan/fitch-proofs

2.0.1 $=$ Intro

1	$a = a$	$=$ *Intro*

2.0.2 $=$ Elim

1	$a = b$	
2	$P(a)$	
3	$P(b)$	$=$ Elim: $2, 1$

Line numbers: first the formula you're changing, then the equality you're using to justify the substitution.

2.1 Boolean connectives

2.1.1 $\wedge$ Intro

1	A	
1	B	
3	$A \wedge B$	$\wedge$ Intro: $1, 2$

2.1.2 $\wedge$ Elim

1	$A \wedge B$	
2	A	$\wedge$ Elim: 1
3	B	$\wedge$ Elim: 1

2.1.3 $\vee$ Intro

1	A	
2	$A \vee B$	$\vee$ Intro: 1

2.1.4 $\vee$ Elim

1	$A \vee B$	
2	$A \to C$	
3	$B \to C$	
4	$\quad A$	
5	$\quad C$	$\to$ Elim: $2, 4$
6	$\quad B$	
7	$\quad C$	$\to$ Elim: $3, 6$
8	C	$\vee$ Elim: 1, 4-5, 6-7

Line numbers: first the line number with the disjunction, followed by ranges for each of the subproofs that corresponds to each disjunct (in order)

2.2 Negation

2.2.1 $\bot$ Intro

1	$\neg A$	
2	A	
3	$\bot$	$\bot$ Intro: $2, 1$

Line numbers: first the positive formula, then the negation of the first line

2.2.2 $\bot$ Elim

Rule: if you get $\bot$, you may conclude anything you want, because the argument is based on false or contradictory assumptions

1	$\bot$	
2	$p = np$	$\bot$ Elim: 1

2.2.3 $\neg$ Intro

1	A	
2	$\quad \neg A$	
3	$\quad \bot$	$\bot$ Intro: $1, 2$
4	$\neg\neg A$	$\neg$ Intro: 2-3

Line numbers: just the subproof range which starts with `P` and ends with $\bot$, then conclude `¬P`

2.2.4 $\neg$ Intro

1	$\neg\neg A$	
2	A	$\neg$ Elim: 1

2.3 (bi-)implication

2.3.1 $\rightarrow$ Intro

1	$a = b$	
2	$\quad P(a)$	
3	$\quad P(b)$	$=$ Elim: $2, 1$

4	$P(a) \to P(b)$	$\to$ Intro: 2-3

2.3.2 $\to$ Elim

1	P	
2	$P \to Q$	
3	Q	$\to$ Elim: $2, 1$

NB: these line numbers are the other way around when compared to `= Elim`; implication first, then the formula on the left-hand side of the implication

2.3.3 $\leftrightarrow$ Intro

Both $\leftrightarrow$ rules are just the same as implications, but both ways

1	$a = b$	
2	$\quad P(a)$	
3	$\quad P(b)$	$=$ Elim: $2, 1$
4	$\quad P(b)$	
5	$\quad a = a$	$=$ Intro
6	$\quad b = a$	$=$ Elim: $5, 1$
7	$\quad P(a)$	$=$ Elim: $4, 6$
8	$P(a) \leftrightarrow P(b)$	$\leftrightarrow$ Intro: 2-3, 4-7

2.3.4 $\leftrightarrow$ Elim

Look mom, it goes left as well! It's almost like the arrow is pointing both ways.

1	$P \leftrightarrow Q$	
2	Q	
3	P	$\leftrightarrow$ Elim: $1, 2$

NB: Once again, this rule's line numbers start with the line with the $\leftrightarrow$, only then the line you're "using".

2.4 Quantifiers

2.4.1 ∀ Intro

1	| | [a]	*For an arbitrary a...*
2	| | $a = a$	= Intro
3	$\forall x(x = x)$	∀ Intro: 1-2

2.4.2 ∀ Elim

1	$\forall x(P(x) \to Q(x))$	
2	$P(a)$	
3	$P(a) \to Q(a)$	∀ Elim: 1
4	$Q(a)$	→ Elim: 3, 2

2.4.3 ∃ Intro

1	$P(a)$	
2	$\exists x P(x)$	∃ Intro: 1

2.4.4 ∃ Elim

1	$\exists x P(x)$	
2	| [a] $P(a)$	*For an arbitrary a for which $P(a)$ holds...*
3	| $P(a) \vee Q(a)$	∨ Intro: 2
4	| $\exists x(P(x) \vee Q(x))$	∃ Intro: 2
5	$\exists x(P(x) \vee Q(x))$	∃ Elim: 1, 2-4

3 Logical Equivalences and laws

These laws are used for NNF, CNF, and DNF. For Prenex, see the section about Prenex.

- Double negation elimination (`NN`): $\neg\neg P \Leftrightarrow P$
- Associativity:
  - $P \wedge (Q \wedge R) \Leftrightarrow (P \wedge Q) \wedge R$
  - $P \vee (Q \vee R) \Leftrightarrow (P \vee Q) \vee R$
- Commutativity:
  - $P \wedge Q \Leftrightarrow Q \wedge P$
  - $P \vee Q \Leftrightarrow Q \vee P$
- Idempotency:

  - $P \wedge P \Leftrightarrow P$
  - $P \vee P \Leftrightarrow P$
- DeMorgan's laws (`DeM`):
  - $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$
  - $\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$
- Definition of $\rightarrow$ (`Def. ->`): $P \rightarrow Q \Leftrightarrow \neg P \vee Q$
- Definition of $\leftrightarrow$ (`Def. <->`):
  - $P \leftrightarrow Q \Leftrightarrow (P \rightarrow Q) \wedge (Q \rightarrow P)$
  - Alternatively, use `Def. ->` immediately. This is allowed on the test, but the above line is more intuitive.
  - $P \leftrightarrow Q \Leftrightarrow (\neg P \vee Q) \wedge (\neg Q \vee P)$

4 Normal forms

In general: use the above laws to achieve the normal forms below. Some tips:

- NNF: usually `(DeM,NN)` is enough
- CNF/DNF: usually you can eliminate a connective that shouldn't be at that "level" with a distributivity law

4.1 Negation normal form (NNF)

Def. a sentence is in NNF if all negations are over atomic sentences, and not compound sentences. Also no $\rightarrow$ or $\leftrightarrow$.

This is fine: $P \wedge (\neg Q \vee R)$

This is not: $P \wedge \neg(Q \vee R)$

4.2 Conjunction normal form (CNF)

Def. a sentence is in CNF if it's NNF, and all conjunctions are above disjunctions.

This is fine: $P \wedge (Q \vee R)$

This is not: $P \vee (Q \wedge R)$

4.3 Disjunction normal form (DNF)

Def. a sentence is in DNF if it's NNF, and all disjunctions are above conjunctions.

This is fine: $P \vee (Q \wedge R)$

This is not: $P \wedge (Q \vee R)$

4.4 Quantifier normal forms

4.4.1 Prenex normal form

Def. a wff is in Prenex normal form if it is in NNF, and no quantifiers under boolean connectives, i.e. all quantifiers "at the top"

Logical laws:

- Renaming: $\forall x P(x) \Leftrightarrow \forall y P(y)$
- DeMorgan:
  - $\neg \forall x P(x) \Leftrightarrow \exists x \neg P(x)$
  - $\neg \exists x P(x) \Leftrightarrow \forall x \neg P(x)$
  - etc.
- Same quantifier:
  - $\forall x P(x) \wedge \forall x Q(x) \Leftrightarrow \forall x (P(x) \wedge Q(x))$
  - $\forall x P(x) \vee \forall x Q(x) \Leftrightarrow \forall x (P(x) \vee Q(x))$
- Implications:
  - These next two assume no free x in P.
  - $P \to \forall x Q(x) \Leftrightarrow \forall x (P \to Q(x))$
  - $\forall x Q(x) \to P \Leftrightarrow \exists x (Q(x) \to P)$

4.4.2 Skolem normal form

Def. a wff is in Skolem normal form if it is in Prenex normal form, without $\exists$ in the formula

Steps:

- Make it Prenex first
- For each $\exists$:
  - $\exists$ in front: make a constant
  - $\exists$ after a $\forall$: make a function that depends on all previous $\forall$'s

For instance:

$$\exists x \forall y \forall z \exists w \forall e \exists g P(x, y, z, w, e, g)$$

becomes

$$\forall y \forall z \forall e P(c, y, z, f_1(y, z), e, f_2(y, z, e))$$

Note: I've used f_1 and f_2 here because I've ran out of letters. Use different letters for the functions preferably.

4.4.3 Horn form

Def. a wff is in Horn form if it is in Prenex form, and the quantifier-free part is in CNF, where each conjunct has at most 1 positive literal.

Usually written as a conjunction of implications, in either of these forms:

- $\top \to P$
- $P \to \bot$
- $P \to Q$ Notice how using `Def ->` on the above forms results in either zero or one positive literals.

4.4.3.1 Horn's algorithm

Horn's algorithm is used for finding satisfiability in a Horn sentence.

Suppose you have the following sentence:

$$\begin{aligned}&(\top \to A)\\ \land\ &(A \to D)\\ \land\ &(C \to \bot)\\ \land\ &((D \land A) \to B)\\ \land\ &((B \land D) \to E)\end{aligned}$$

Apply the following steps:

- Make a table (example below), for each of the atomic sentences, and $\bot$.
- Either until you can't anymore, or you've put a `T` in the column for $\bot$, analyse the consequences of the values you've added before
- If you have no more information, but a `F` in all remaining columns.
- When you're done, decide whether the sentence is satisfiable:
  - $\bot = T$: *not satisfiable*
  - otherwise: *satisfiable*

For this sentence, you get:

	A	**B**	**C**	**D**	**E**	$\bot$
truth value	T	T	F	T	T	F
step	1	3		2	4	

5 Models

5.1 Interpretations

Models are used to interpret a FO-language, and describe a situation in that language. We will define a FO-language $\mathcal{L}$ in this section, to show how each of the elements can be represented in a model.

Let $\mathcal{L}$ be a first-order language with the following elements:

- constant ah: Albert Heijn
- constant $plus$: Plus supermarket
- unary function $ceo(x)$: the CEO of x
- unary predicate $Best(x)$: x is the best
- binary predicate $Better(x, y)$: x is better than y

Now let's describe the following situation:

- The CEO of Albert Heijn is Margaret
- The CEO of Plus is Aart
- Albert Heijn is better than Plus
- Aart is the best CEO ever

So, let's now define a first-order model $\mathfrak{M}$:

- $\mathfrak{M}(\forall) = \{\text{Albert Heijn}, \text{Plus}, \text{Aart}, \text{Margaret}\}$ (this is our domain of discourse)
- $\mathfrak{M}(ah) = \text{Albert Heijn}$ (interpretation of a constant)
- $\mathfrak{M}(plus) = \text{Plus}$
- $\mathfrak{M}(ceo)(\text{Albert Heijn}) = \text{Margaret}$ (interpretation of a function)
- $\mathfrak{M}(ceo)(\text{Plus}) = \text{Aart}$
- $\mathfrak{M}(Best) = \{\text{Aart}\}$ (interpretation of a unary predicate is a set)
- $\mathfrak{M}(Better) = \{\langle \text{Albert Heijn}, \text{Plus}\rangle\}$ (interpretation of a binary predicate is a relation)

5.2 Variable assignments

Special notation in the segment below, used to specify what the values of variables are (x,y,z, etc.)

The test will define variable assignments as such: let h be a variable assignment such that $h(x) = 69$ and $h(y) = 420$.

Notations:

- $h(x)$: the value of x in the variable assignment h
- $h[x/d]$: a copy of h, where the variable x is overwritten with value d
- $h_{\varnothing}$: empty variable assignment
- $\mathfrak{M} \vDash P[h]$: the model satisfies P under variable assignment h

5.3 Proving satisfiability in a model

For this summary, I will write down the equivalences you'll need to memorize. Practice applying them with the various model exercises that are available.

5.3.1 Atomics

- Unary predicates:
  - $\mathfrak{M} \vDash P(x)[h]$ iff $[\![x]\!]_h^{\mathfrak{M}} \in \mathfrak{M}(P)$
  - $\mathfrak{M} \nvDash P(x)[h]$ iff $[\![x]\!]_h^{\mathfrak{M}} \notin \mathfrak{M}(P)$
- Binary predicates:
  - $\mathfrak{M} \vDash Q(x,y)[h]$ iff $\langle [\![x]\!]_h^{\mathfrak{M}}, [\![y]\!]_h^{\mathfrak{M}}\rangle \in \mathfrak{M}(Q)$
  - $\mathfrak{M} \nvDash Q(x,y)[h]$ iff $\langle [\![x]\!]_h^{\mathfrak{M}}, [\![y]\!]_h^{\mathfrak{M}}\rangle \notin \mathfrak{M}(Q)$

5.3.2 Boolean connectives

- $\mathfrak{M} \vDash (P \wedge Q)[h]$ iff $\mathfrak{M} \vDash P[h]$ and $\mathfrak{M} \vDash Q[h]$
- $\mathfrak{M} \vDash (P \vee Q)[h]$ iff $\mathfrak{M} \vDash P[h]$ or $\mathfrak{M} \vDash Q[h]$
- $\mathfrak{M} \vDash \neg P[h]$ iff $\mathfrak{M} \nvDash P[h]$
- $\mathfrak{M} \vDash (P \rightarrow Q)[h]$ iff $\mathfrak{M} \nvDash P[h]$ or $\mathfrak{M} \vDash Q[h]$
- $\mathfrak{M} \vDash (P \leftrightarrow Q)[h]$ iff $(\mathfrak{M} \vDash P[h]$ and $\mathfrak{M} \vDash Q[h])$ or $(\mathfrak{M} \nvDash P[h]$ and $\mathfrak{M} \nvDash Q[h])$

5.3.3 Quantifiers

- $\mathfrak{M} \vDash \forall x P(x)[h]$ iff for all $d \in \mathfrak{M}(\forall) : \mathfrak{M} \vDash P(x)[h[x/d]]$
- $\mathfrak{M} \vDash \exists x P(x)[h]$ iff for some $d \in \mathfrak{M}(\forall) : \mathfrak{M} \vDash P(x)[h[x/d]]$